Cálculo del Área de un Círculo con Diseño Top-Down

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 3 — Enfoque modular y diseño Top-Down **Fecha:** 2026

Introducción

Esta actividad aplica un enfoque **estructurado y Top-Down** al diseño de un programa. En lugar de escribir un bloque lineal de código, el problema se descompone en módulos independientes, cada uno con una responsabilidad única.

El problema concreto es calcular el **área de un círculo** a partir de su radio, usando la fórmula matemática `A = π · r²`. Sobre ese núcleo se agrega un bonus de gamificación: la consola simula una **diana de tiro en ASCII** cuyo tamaño depende del radio ingresado.

La pregunta que guía el desarrollo es qué aporta realmente modularizar un programa de veinte líneas, y la respuesta resulta más interesante de lo que parece: un módulo bien definido no solo facilita la lectura: cambia la forma en que se piensa el problema.

Desarrollo

Diseño Top-Down

El diseño Top-Down consiste en tomar un problema complejo de ingeniería y dividirlo en partes o módulos más pequeños y fáciles de resolver. En C++ esto se logra mediante la creación de **funciones diseñadas por el programador**.

El beneficio concreto de modularizar es triple:

• **Depuración**: los errores se aíslan dentro de un módulo y se encuentran más

rápido.

• **Eficiencia**: el código se reutiliza sin duplicarlo.

• **Trabajo en equipo**: cada persona puede desarrollar un modulo distinto sin

pisarse con el resto.

En esta actividad la descomposición fue:

```
Programa principal (main)
■■■ Solicita el radio al usuario
■■■ Valida el dato (filtro de consistencia)
■■■ Llama al módulo calcularAreaCirculo()
■■■ Muestra el resultado y la diana

Módulo calcularAreaCirculo(radio)
■■■ Aplica A = π · r² y devuelve el área
```

El prototipo de la función

El **prototipo de función** es la declaración de firma que se escribe antes del bloque `main()`. Le avisa al compilador de C++ el nombre de la función, qué tipo de dato va a devolver y qué parámetros necesita para ejecutarse.

```
double calcularAreaCirculo(double radio);
```

C++ compila el archivo **de arriba hacia abajo**. Si `main()` invoca la función antes de que el compilador haya leído su desarrollo completo, el compilador no sabe qué hacer y arroja un error. El prototipo resuelve ese problema de orden de lectura sin obligar a mover el desarrollo de la función por encima del `main`.

La llamada y el retorno

Con la firma declarada, la invocación desde `main()` es directa:

```
double areaFinal = calcularAreaCirculo(radioEstudiante);
```

Y la función devuelve el resultado con `return`:

```
double calcularAreaCirculo(double r)
{
    const double PI = 3.1415926535;
    double area = PI * r * r;
    return area;
}
```

El **retorno** (*return*) es el valor o resultado final que la función le devuelve al programa principal una vez que termina de realizar sus cálculos internos.

Proteger la constante PI en la RAM

El número π es un valor fijo universal que no debe cambiar jamás. En C++, declarar `const double PI = 3.1415926535;` le indica al compilador que **bloquee ese casillero de memoria**: cualquier intento posterior de reasignarlo produce un error de compilación.

Esto no es una cuestión estética. Si `PI` fuera una variable común, cualquier línea de código posterior podría modificarla por error y todos los calculos que ya se hubieran hecho quedarian invalidados en silencio, sin ningún aviso. La constante convierte un error difícil de detectar en un error que el compilador detecta inmediatamente.

El filtro de consistencia

```
if (radioEstudiante <= 0)
{
    cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
    return 1;
}
```

Este es el **filtro de consistencia** de la actividad: la condición lógica que el programador escribe para evitar que el usuario ingrese datos imposibles.

Físicamente, un círculo de radio `0 cm` es un punto, que no tiene área. Un radio de `-5 cm` es directamente una Magnitud imposible. Geométricamente, $A = \pi \cdot r^2$ con $r = 0$ da un área de 0, que es un resultado formalmente calculable pero geométricamente inválido: la fórmula asume que el radio describe una circunferencia real, y 0 y los negativos no lo hacen. Controlarlo evita que el programa entregue un número sin sentido físico como si fuera una respuesta válida.

El bonus: la diana en ASCII

Como recompensa al diseño modular correcto, la consola simula un blanco de tiro según el tamaño del radio, usando tres escalas distintas (mini, estándar y gigante). El `if / else if / else` se apoya en la misma variable `radioEstudiante` que ya fue validada, de modo que el bonus no necesita ninguna validación adicional.

Conclusión

La actividad demostró que el diseño Top-Down no es solo una forma ordenada de escribir código, sino una forma de **pensar el problema**. Al separar la validación, el cálculo y la presentación en módulos con responsabilidades claras, cada uno se puede probar, reutilizar y modificar sin romper el resto.

Las tres ideas centrales quedan consolidadas: el **prototipo** resuelve el problema de orden de lectura del compilador; el **retorno** es el canal por el que un módulo entrega su resultado al programa principal; y la palabra clave `const` protege los datos que no admiten cambio, convirtiendo un error silencioso en un error de compilación.

En un equipo real de desarrollo, el diseño modular no es un capricho académico: es lo que permite que cinco personas trabajen el mismo proyecto sin pisarse, y lo que hace que un error se localice en minutos en lugar de horas.

Referencias

• The Apache Software Foundation. (2024). *Structure of a program*. The C++ Book.
<https://the-cpp-book.github.io/>

• cppreference.com. (2024). *Namespaces*. C++ reference.
<https://en.cppreference.com/w/cpp/language/namespace>

• Microsoft. (2024). *const (C++)*. Microsoft Learn.
<https://learn.microsoft.com/es-es/cpp/cpp/const-cpp>

• Microsoft. (2024). *Functions (C++)*. Microsoft Learn.
<https://learn.microsoft.com/es-es/cpp/cpp/functions-cpp>

• Stewart, I. (2023). *Circle area*. Wikipedia.
<https://en.wikipedia.org/wiki/Circle